



View records

Views are what define how records should be displayed to end-users. They are specified in XML and stored as records themselves, meaning they can be edited independently from the models that they represent. They are flexible and allow a high level of customization of the screens that they control. There exist various **types of views**. Each represents a visualization mode: *form*, *list*, *kanban*, etc.

Generic structure

Basic views generally share the common minimal structure defined below. Placeholders are denoted in all caps.

```
<record id="ADDON.MODEL_view_TYPE" model="ir.ui.view">
  <field name="name">NAME</field>
  <field name="model">MODEL</field>
  <field name="arch" type="xml">
    <VIEW_TYPE>
      <views/>
    </VIEW_TYPE>
  </field>
</record>
```

View types

Form

Display and edit the data from a single record.

List

View and edit multiple records.

Search

Apply filters and perform searches. The results are displayed in the current list, kanban... view.

Kanban

Display records as "cards", configurable as a small template.

Qweb

Templating of reporting, website...

Display aggregations as a [pivot table](#).

Calendar

Display records as events in a daily, weekly, monthly, or yearly calendar.

Cohort Enterprise feature

Display and understand the way some data changes over a period of time.

Gantt Enterprise feature

Display records as a Gantt chart.

Grid Enterprise feature

Display computed information in numerical cells; are hardly configurable.

Map Enterprise feature

Display records on a map, and the routes between them.

Fields

View records expose a number of fields.

`class odoo.addons.base.models.ir_ui_view.IrUIView` [\[source\]](#)

name

Only useful as a mnemonic/description of the view when looking for one in a list of some sort. Most Odoo view names start with the name of the addon and end with the type of view being discussed.

Requirement:

Optional

Type:

[Char](#)

model

The model linked to the view, if applicable.

Requirement:

Mandatory

Type:

[Char](#)

arch

Optional

Type:

Text

groups_id

The groups allowed to use/access the current view.

If the view extends an existing view, the extension will be applied only for a given user, if that user has access to the provided `groups_id`.

Requirement:

Optional

Type:

Many2many → Groups

priority

When requesting a view by specifying the `model` and `type`, the matching view with the lowest priority is returned (it is the default view).

It also defines the order of views application during [view resolution](#). When a view is requested by `id` and its mode is not `primary`, its *closest* parent with `mode = primary` is matched.

Requirement:

Optional

Type:

Integer

inherit_id

Reference to the parent view on which the [inheritance](#) will be applied. Its value is used by default. Specify the parent using the `ref` attribute with `ref="ADDON.MODEL_parent_view_TYPE"`.

The addon name (before the dot) is not necessary if the inheritance is done on a record of the same module.

See [Inheritance](#) for more information.

Requirement:

mode

Only applies if this view inherits from an other one (`inherit_id` is set).

extension

If the view is requested, the closest primary view is looked up (via `inherit_id`). Then, all views inheriting from it with this view's model are applied.

primary

The closest primary view is fully resolved (even if it uses a different model than the current one). Then, the view's **inheritance specs** are applied, and the result is used as if it were this view's actual arch.

A case in which one would want to override `mode` while using `inherit_id` is delegation inheritance. In that case, your derived model is separated from its parent, and views matching with one won't match with the other. Assuming one inherits from a view associated with the parent model and wants to customize the derived view to show data from the derived model, the `mode` of the derived view needs to be set to `primary` because it is the base (and maybe only) view for that derived model. Otherwise, the **view matching** rules won't apply.

See **Inheritance** for more information.

Requirement:

Optional

Type:

Selection : `extension` / `primary`

Default:

`extension`

Note

The current context and user access rights may also impact the view abilities.

Inheritance

Inheritance allows for customizing delivered views. It makes it possible, for example, to add content as modules are installed, or to deliver different displays according to the action.

```
<record id="ADDON.MODEL_view_TYPE" model="ir.ui.view">
  <field name="model">MODEL</field>
  <field name="inherit_id" ref="VIEW_REFERENCE"/>
  <field name="mode">MODE</field>
  <field name="arch" type="xml">
    <xpath expr="XPATH" position="POSITION">
      <CONTENT/>
    </xpath>
    <NODE ATTRIBUTES="VALUES" position="POSITION">
      <CONTENT/>
    </NODE>
  </field>
</record>
```

The `inherit_id` and `mode` fields determine the **view resolution**. The `xpath` or `NODE` elements indicate the **inheritance specs**. The `expr` and `position` attributes specify the **inheritance position**.

View resolution

Resolution generates the final `arch` for a requested/matched primary view as follow:

1. if the view has a parent, the parent is fully resolved, then the current view's inheritance specs are applied;
2. if the view has no parent, its `arch` is used as-is;
3. the current view's children with `mode extension` are looked up, and their inheritance specs are applied depth-first (a child view is applied, then its children, then its siblings).

The inheritance is applied according to the `inherit_id` field. If several view records inherit the same view, the order is determined by the `priority`.

The result of applying children views yields the final `arch`.

Inheritance specs

Inheritance specs are applied sequentially and are comprised of:

1. an element locator to match the inherited element in the parent view;
2. children element to modify the inherited element.

- A `field` element with a `name` attribute, matching the first field with the same `name` .

Note

All other attributes are ignored.

- Any other element, matching the first element with the same `name` and identical attributes.

Note

The attributes `position` and `version` are ignored.

[1] An extension function is added for simpler matching in QWeb views: `hasclass(*classes)` matches if the context node has all the specified classes.

Example

```
<xpath expr="page[@name='pg']/group[@name='gp']/field" position="inside">
    <field name="description"/>
</xpath>

<div name="name" position="replace">
    <field name="name2"/>
</div>
```

Inheritance position

The inheritance specs accept an optional `position` attribute, defaulting to `inside` , that specifies how the matched node should be modified.

inside

The content of the inheritance spec is appended to the matched node.

Example

```
</page>
</notebook>
```

after

The content of the inheritance spec is appended to the matched node's parent after the matched node.

Example

```
<xpath expr="//field[@name='x_field']" position="after">
  <field name="x_other_field"/>
</xpath>
```

before

The content of the inheritance spec is appended to the matched node's parent before the matched node.

Example

```
<field name=x_field" position="before">
  <field name="x_other_field"/>
</field>
```

replace

The content of the inheritance spec replaces the matched node. Any text node containing only `$0` within the contents of the spec is replaced by a copy of the matched node, effectively wrapping the matched node.

Example

```
<xpath expr="//field[@name='x_field']" position="replace">
  <div class="wrapper">
    $0
  </div>
</xpath>
```

attributes

The content of the inheritance spec should be made of only `attribute` elements, each with a `name` attribute and an optional body.

- If the `attribute` element has a body, a new attributed named after its `name` is added to the matched node with the `attribute` element's text as value.
- If the `attribute` element has no body, the attribute named after its `name` is removed from the matched node.
- If the `attribute` element has an `add` attribute, a `remove` attribute, or both, the value of the matched node's attribute named after `name` is recomputed to account for the value(s) of `add`, `remove`, and an optional `separator` attribute defaulting to `,`. `add` includes its value(s), separated by `separator`. `remove` removes its value(s), separated by `separator`.

Example

```
<field name="x_field" position="attributes">
  <attribute name="invisible">True</attribute>
  <attribute name="class" add="mt-1 mb-1" remove="mt-2 mb-2" separator=" "/>
</field>
```

move

The attribute `position="move"` is set on the content of the inheritance spec to specify how nodes are moved relatively to the inheritance spec's element locator, on which the attribute `position` must also be set, with values `inside`, `replace`, `after`, or `before`.

Example

```
<xpath expr="//@target" position="after">
  <xpath expr="//@node" position="move"/>
</xpath>

<field name="target_field" position="after">
```


Model commons

`class odoo.addons.base.models.ir_ui_view.Base` [\[source\]](#)

`get_views(views, options=None)` [\[source\]](#)

Returns the fields_views of given views, along with the fields of the current model, and optionally its filters for the given action.

The return of the method can only depend on the requested view types, access rights (views or other records), view access rules, options, context lang and TYPE_view_ref (other context values cannot be used).

Python expressions contained in views or representing domains (on python fields) will be evaluated by the client with all the context values as well as the record values it has.

Parameters:

views – list of [view_id, view_type]

options (*dict*) –

a dict optional boolean flags, set to enable:

toolbar

includes contextual actions when loading fields_views

load_filters

returns the model's filters

action_id

id of the action to get the filters, otherwise loads the global filters or the model

Returns:

dictionary with fields_views, fields and optionally filters

`get_view([view_id | view_type='form'])` [\[source\]](#)

Get the detailed composition of the requested view like model, view architecture.

The return of the method can only depend on the requested view types, access rights (views or other records), view access rules, options, context lang and TYPE_view_ref (other context values cannot be used).

Parameters:

view_id (*int* or *None*) – id of the view or None

param bool mobile:

true if the web client is currently using the responsive mobile view (to use kanban views instead of list views for x2many fields)

Returns:

composition of the requested view (including inherited views and extensions)

Return type:

[dict](#) 

Raises:

[AttributeError](#)  –

if the inherited view has unknown position to work with other than 'before', 'after', 'inside', 'replace'
if some tag other than 'position' is found in parent view

 Get Help[Contact Support](#)[Ask the Odoo Community](#)